

3. Пример расчета LRC8 с помощью бинарных сдвиговых операций.
4. Исходные тексты разработанных программ.
5. Перечень используемых полиномов.
6. Все модификации "секретного" сообщения и соответствующие значения контрольных сумм и CRC в процессе выполнения работы.
7. Результаты анализа свойств используемых способов расчета контрольных сумм и CRC по отношению к воздействиям на данные.
8. Результаты расчета CRC для бинарного файла.
9. Результаты расчета CRC для программы, реализующей сдвиговые операции.
10. Ответы на контрольные вопросы.
11. Выводы по работе и предложения, какие изменения необходимо внести в разработанные программы, чтобы улучшить их быстродействие.

Контрольные вопросы

1. В каких случаях с помощью контрольной суммы можно обнаружить подмену или искажение информации?
 2. Почему при подсчете CRC применяется полиномиальная арифметика по модулю 2?
 3. Какие ошибки CRC могут возникнуть при передаче пакета?
 4. Почему для расчета CRC применяют определенные полиномы?
 5. Можно ли, зная CRC, исправить ошибку при передаче текста?
- Ответ обосновать.

Лабораторная работа 10 **МЕТОДЫ НАДЕЖНОГО ХРАНЕНИЯ И ПЕРЕДАЧИ ИНФОРМАЦИИ**

Цель работы – ознакомление с методом Хемминга помехоустойчивого кодирования, позволяющим обнаруживать и автоматически исправлять ошибки, возникающие при хранении и передаче информации в ненадежных средах.

Теоретические сведения

Среда, в которой передается, обрабатывается и хранится информация, не может быть абсолютно надежной. В беспроводных системах связи и телекоммуникационных системах уровень помех бывает очень высоким. Наиболее простой способ проверки целостности передаваемых данных – использование контрольных сумм и CRC. Однако такой подход позволяет только обнаруживать влияние среды на передаваемую информацию. В том случае, если необходимо избежать повторной передачи данных и исправить ошибки в уже полученных данных, применяется помехоустойчивое кодирование.

Теорема Шеннона утверждает, что при любой производительности источника данных, меньшей, чем пропускная способность канала, существует такой способ кодирования, который обеспечивает передачу всей информации со сколь угодно малой вероятностью ошибки.

Методы помехоустойчивого кодирования характеризуются использованием избыточной информации и усреднением шума. Закодированные помехоустойчивые сообщения всегда содержат дополнительные (избыточные) символы, поэтому помехоустойчивое кодирование также называют избыточным. Дополнительные символы применяют для того, чтобы уменьшить вероятность потери сообщением своей индивидуальности из-за возможных искажений. Эффект усреднения шума достигается за счет того, что избыточные символы зависят от нескольких значимых информационных символов.

В результате влияния помех в сообщении возникают однокбитные или групповые ошибки. У групповых ошибок есть свои позитивные и негативные свойства. Положительное свойство групповой ошибки заключается в следующем. Пусть данные передаются блоками по 1000 бит, вероятность ошибки – 0,001 на бит. Если ошибки изолированы и независимы, то 63% блоков ($0,63 \approx 1 - 0,999^{1000}$) содержат ошибки. Если они возникают группами по 100, то ошибки содержат 1% блоков ($0,01 \approx 1 - 0,999^{10}$). Зато, если ошибки не группируются, то в каждом кадре они невелики, и есть возможность их исправить.

Негативное свойство групповых ошибок – безвозвратный урон, наносимый сообщению, в результате чего требуется повторная пересылка, что в некоторых системах невозможно (например, в телефонных сетях).

В помехоустойчивом кодировании вводится понятие расстояния Хемминга – минимального числа разрядов с неодинаковыми значениями кодовых слов. Например, последовательность двоичных кодов $\{0_{(10)}=00_{(2)}, 1_{(10)}=01_{(2)}, 2_{(10)}=10_{(2)}, 3_{(10)}=11_{(2)}\}$ имеет расстояние Хемминга, равное единице, поскольку два любых числа отличаются, по меньшей мере, на один бит. В таком коде ошибки невозможно обнаружить. Последовательность кодов $\{0_{(10)}=00_{(2)}, 1_{(10)}=11_{(2)}\}$ имеет расстояние Хемминга, равное двум, и позволяет обнаруживать одиночные ошибки. Комбинации $\{01_{(2)}, 10_{(2)}\}$ называются запрещенными. Если они встречаются декодером, то это служит признаком ошибки в переданном кодовом слове. Таким образом, за счет сокращения информационной емкости кодов становится возможным обнаруживать нарушения целостности передаваемой информации. Для исправления одного сбойного бита требуется увеличить расстояние Хемминга до трех. При этом информационная емкость таких кодов существенно падает, например, четырехразрядный код с расстоянием Хемминга, равным трем, способен вместить только два информационных символа.

Обнаружив ошибку, декодер последовательно сравнивает искаженный символ со всеми разрешенными символами, стремясь найти символ, наиболее схожий с искаженным (то есть символ с наименьшим числом различий, а именно не более, чем в $d-1$ позициях, где d – расстояние Хемминга). Например, 10-разрядный код $\{0000000000, 0000011111, 1111100000, 1111111111\}$ имеет расстояние Хемминга, равное пяти ($d = 5$). Он позволяет обнаруживать четырехбитные ошибки ($d-1 = 4$) и гарантированно исправлять двубитные ($(d-1)/2 = 2$). Если вместо символа 0000011111 получен символ 0001010111, то ближайший символ – 0000011111. Если вместо двух бит искажение затрагивает максимально допустимое для восстановления количество разрядов – четыре, то восстановление декодером не гарантируется. Например, если бы вместо 0000011111 декодер получил код 0111111111, то он восстановил бы символ 1111111111 по наименьшему расстоянию.

В общем случае, если количество сбойных бит меньше расстояния Хемминга хотя бы наполовину, то декодер может гарантированно восстановить исходный символ.

Условие обнаружения ошибок: $d \geq r$, где r – количество ошибок. Условие успешного восстановления информации: $d > 2r$. Поэтому простейшая реализации кодирования Хемминга дополнительно к информационным разрядам вводит $L = \log_2 K$ избыточных контролирующих разрядов, где K – число информационных разрядов. Число L округляется до ближайшего большего целого значения. L -разрядный контролирующий код есть инвертированный результат поразрядного сложения номеров тех информационных разрядов, значения которых равны 1. Например, имеется сообщение 100110. Число информационных разрядов $K = 6$. Следовательно, число избыточных битов $L = 3$. Номера единичных битов: 2, 3 и 6. Дополнительный код равен $2_{(10)} + 3_{(10)} + 6_{(10)} = 010_{(2)} + 011_{(2)} + 110_{(2)} = 111_{(2)}$, где "+" обозначает поразрядное сложение. После инвертирования бинарный код принимает вид 000. Декодер рассчитывает избыточный код и сравнивает с переданным значением. Затем он фиксирует код сравнения (поразрядная операция отрицания равнозначности), и, если результат отличен от нуля, то его значение – это номер ошибочно принятого разряда основного кода. Например, если принят код 100010, то рассчитанный декодером дополнительный код равен инверсии суммы $010_{(2)} + 110_{(2)} = 100_{(2)}$, то есть $011_{(2)}$, что означает ошибку в третьем разряде.

Теоретически допустимое количество исправляемых ошибок не ограничено, однако, практически информационная емкость кодовых слов уменьшается с ростом расстояния Хемминга. Поэтому приведенная схема кодирования усовершенствуется за счет того, что каждый контрольный бит отвечает за четность суммы некоторой сопоставленной ему группы битов, причем один бит может относиться к разным группам. Таким образом, один информационный бит влияет на несколько контрольных, и информационная емкость слова значительно возрастает.

Согласно методу Хемминга, для определения номеров позиций битов, которые контролируют информационный бит, стоящий в позиции k , необходимо разложить k по степеням двойки (табл. 12). С ростом разрядности кодового слова контрольные биты располагаются реже, поэтому с увеличением разрядности обрабатываемого слова эффективность кодов Хемминга значительно возрастает.

Таблица 12

Схема размещения информационных и контрольных бит

Номер бита	Разложение номера бита по степеням	Тип бита
1	2^0	Контрольный (контролирует биты 3, 5 и 7)
2	2^1	Контрольный (контролирует биты 3, 6 и 7)
3	$2^0+2^1 = 1+2$	Информационный (контролируется битами 1 и 2)
4	2^2	Контрольный (контролирует биты 5, 6 и 7)
5	$2^0+2^2 = 1+4$	Информационный (контролируется битами 1 и 4)
6	$2^1+2^2 = 2+4$	Информационный (контролируется битами 2 и 4)
7	$2^0+2^1+2^2 = 1+2+4$	Информационный (контролируется битами 1, 2 и 4)
8	2^3	Крайний бит в слове, не контролирует другие биты

Биты, номера которых являются степенью 2, становятся контрольными, остальные биты – информационными. Каждый контрольный бит отвечает за четность суммы связанной с ним группы бит, расположенных на определенных позициях справа. Если сумма соответствующих информационных битов четная, то контрольный бит равен нулю, и наоборот. Например, в слове 0100 отметим символом * позиции контрольных битов **0*100*. Бит 1, контролирующий биты 3, 5 и 7, равен единице, так как сумма соответствующих информационных битов нечетная ($0+1+0$). Бит 2, контролирующий биты 3, 6 и 7, равен нулю, так как сумма информационных битов четная ($0+0+0$). Бит 4, контролирующий биты 5, 6 и 7, равен единице, так как сумма информационных битов нечетная ($1+0+0$). Битом 8 можно пренебречь, так как он не несет информационной составляющей и ничего не контролирует. Итоговое значение – 1001100. Исказим, например, один бит 1011100. Бит 1 становится равным нулю, бит 2 – единице, бит 4 сохраняется. Контрольные биты 1 и 2 не совпадают с расчетными значениями. Сумма позиций этих битов дает позицию бита ошибки ($1+2=3$). Исправление ошибки достигается путем инверсии бита 3 в полученном коде.

Коды Хемминга способны исправлять лишь одиночные ошибки. С ростом размера обрабатываемых слов увеличивается и вероятность ошибок. Выбор оптимального размера кодируемых слов является нетривиальной задачей, требующей знания характера и частоты возникновения ошибок. Для исправления групповых ошибок применяются более сложные методы, например, кодирование Рида-Соломона.

Порядок выполнения работы

Перед выполнением лабораторной работы необходимо вручную рассчитать код Хемминга для произвольного 16-разрядного числа.

1. Создать текстовый файл размером 100 символов, сохранить его как эталон.

2. Реализовать программу, выполняющую для произвольного текстового файла функции кодирования и декодирования Хемминга. Размер текстового блока, обрабатываемого в рамках одного цикла кодирования, должен задаваться в виде аргумента командной строки.

3. Закодировать текстовый файл. Зафиксировать и сравнить размеры исходного и закодированного файлов для различных размеров кодируемых блоков: 8, 12, 16, 24, 32, 48, 64 разряда.

4. С помощью HEX-редактора (например, *HIEW*) в произвольных местах закодированного файла внести одиночные изменения и зафиксировать позицию в файле и исходное значение измененного бита, чтобы впоследствии можно было проверить данные при декодировании.

5. Запустить разработанную программу на декодирование модифицированного закодированного файла. Сопоставить содержимое декодированного файла с эталоном, отметить совпадения и расхождения. Проверить восстановление измененных битов при декодировании.

6. С помощью HEX-редактора в произвольных местах закодированного файла внести групповые изменения. Данные изменения моделируют массовые ошибки при передаче или хранении информации. Для внесенных изменений необходимо зафиксировать позицию в файле и новое значение измененных битов, чтобы впоследствии можно было проверить факт восстановления данных при декодировании файла. Повторить выполнение п. 5.

Содержание отчета

1. Пример расчета кода Хемминга для 16-разрядного числа.
2. Блок-схема алгоритма вычисления помехоустойчивого кода.
3. Описание однобитных и групповых ошибок, внесенных в файл.
4. Результаты восстановления информации.
5. График зависимости изменения объема закодированной информации от размера кодируемых блоков.
6. Листинг разработанной программы.
7. Ответы на контрольные вопросы.
8. Выводы по работе.

Контрольные вопросы

1. Что такое расстояние Хемминга?
2. Опишите метод кодирования Рида-Соломона.
3. Как контролировать ошибку в бите 8 (табл. 12)?
4. На что влияет размер кодируемого блока в методе Хемминга?
5. Постройте схему размещения информационных и контрольных бит в блоке размером 16 разрядов.

Лабораторная работа 11

МЕХАНИЗМ АУТЕНТИФИКАЦИИ ПОЛЬЗОВАТЕЛЕЙ

Ц е л ь р а б о т ы – получение навыков защиты информационных систем путем проверки подлинности пользователей, исследование проблемы подмены механизма аутентификации в операционной системе Windows.

Теоретические сведения

В защищенной системе каждый пользователь должен быть известен и представлен в виде зарегистрированной системной сущности. В начале рабочего сеанса пользователь должен подтвердить свою идентичность системному представлению путем ввода своих учетных данных (логина и пароля). В этой связи проверка пользователя включает два этапа: